

Department of Computer Science and Engineering

III/II CSE - R23 Regulations
BIG DATA ANALYTICS

UNIT III

Hive, data types and file formats, HiveQL data definition, HiveQL data manipulation, Logical joins, Window functions, Optimization, Table partitioning, Bucketing, Indexing, Join strategies.

Create Department Table

```
CREATE TABLE department(  
dept_id INT,  
dept_name STRING  
);
```

Data Insertion

```
INSERT INTO employee VALUES  
(101,'Ravi',1,50000),  
(102,'Anil',2,60000),  
(103,'Sita',3,45000),  
(104,'John',2,55000),  
(105,'Priya',1,52000);
```

Insert Data into Department Table

```
INSERT INTO department VALUES  
(1,'HR'),  
(2,'IT'),  
(4,'Finance');
```

Display Employee Records

```
SELECT * FROM employee;
```

Display Selected Columns

```
SELECT name, salary  
FROM employee;
```

Filter Records

```
SELECT *
```

```
FROM employee
WHERE salary > 50000;
```

Count Employees

```
SELECT COUNT(*)
FROM employee;
```

Average Salary

```
SELECT AVG(salary)
FROM employee;
```

HiveQL Join Operations

INNER JOIN

Returns matching rows from both tables.

```
SELECT e.emp_id, e.name, d.dept_name
FROM employee e
JOIN department d
ON e.dept_id = d.dept_id;
```

INNER JOIN with Condition

```
SELECT e.name, d.dept_name, e.salary
FROM employee e
JOIN department d
ON e.dept_id = d.dept_id
WHERE e.salary > 50000;
```

LEFT OUTER JOIN

Returns all rows from employee table.

```
SELECT e.emp_id, e.name, d.dept_name
FROM employee e
LEFT OUTER JOIN department d
ON e.dept_id = d.dept_id;
```

Find Employees without Department

```
SELECT e.name
FROM employee e
LEFT JOIN department d
ON e.dept_id = d.dept_id
WHERE d.dept_id IS NULL;
```

RIGHT OUTER JOIN

Returns all rows from department table.

```
SELECT e.name, d.dept_name
FROM employee e
RIGHT OUTER JOIN department d
ON e.dept_id = d.dept_id;
```

FULL OUTER JOIN

Returns all rows from both tables.

```
SELECT e.emp_id, e.name, d.dept_name  
FROM employee e  
FULL OUTER JOIN department d  
ON e.dept_id = d.dept_id;
```

CROSS JOIN

Produces Cartesian product.

```
SELECT *  
FROM employee  
CROSS JOIN department;
```

SELF JOIN

Find employees working in the same department.

```
SELECT a.name AS Employee1, b.name AS Employee2  
FROM employee a  
JOIN employee b  
ON a.dept_id = b.dept_id  
WHERE a.emp_id <> b.emp_id;
```

Aggregation with Joins

Department-wise Total Salary

```
SELECT d.dept_name, SUM(e.salary) AS total_salary  
FROM employee e  
JOIN department d  
ON e.dept_id = d.dept_id  
GROUP BY d.dept_name;
```

Department-wise Average Salary

```
SELECT d.dept_name, AVG(e.salary) AS avg_salary  
FROM employee e  
JOIN department d  
ON e.dept_id = d.dept_id  
GROUP BY d.dept_name;
```

Window Functions

Rank Employees by Salary

```
SELECT name, salary,  
RANK() OVER (ORDER BY salary DESC) AS salary_rank  
FROM employee;
```

Department-wise Ranking

```
SELECT name, dept_id, salary,  
ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS  
dept_rank  
FROM employee;
```